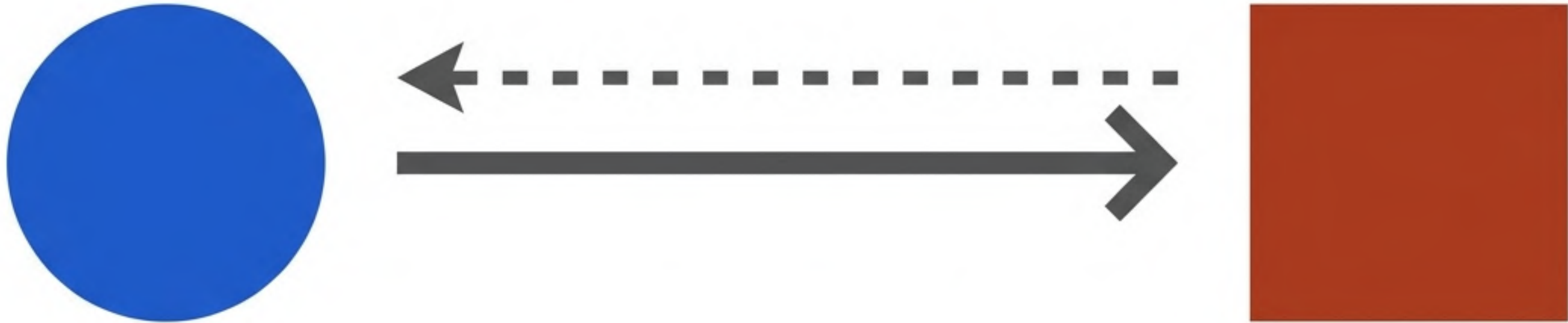


Consumo de datos desde una API: Fundamentos antes de Vue

Antes de consumir datos en Vue, debemos entender cómo viajan los datos entre el cliente y el servidor.



¿Qué es una API?

Interfaz de Programación de Aplicaciones: Un conjunto de reglas que permite que dos sistemas de software se comuniquen entre sí, sin importar su lenguaje.

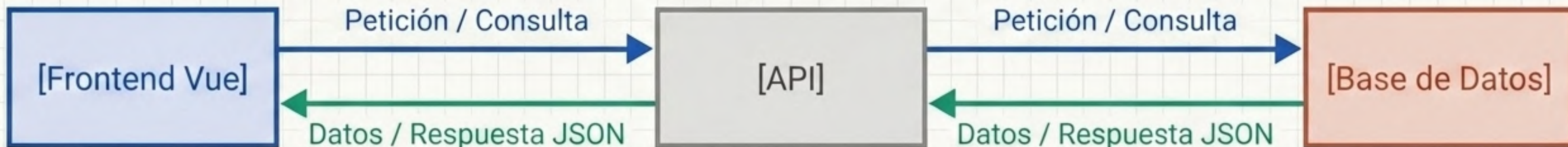
La Analogía



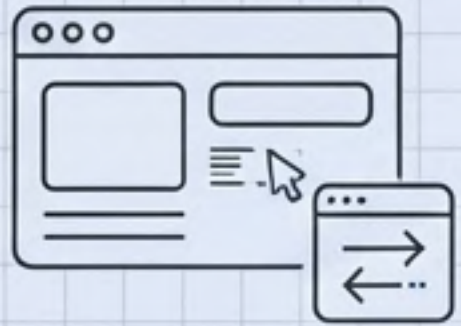
Tu app del clima le hace una pregunta a un servicio externo para saber si lloverá, ya que no tiene termómetros propios.

La API no es la base de datos. Normalmente funciona como una capa intermedia que define qué datos se pueden consultar, crear, actualizar o eliminar.

El Flujo Técnico

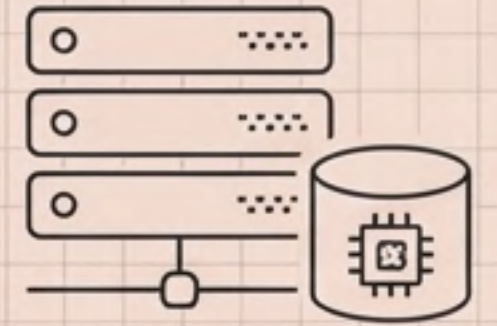


El ciclo de vida de la web: Cliente y Servidor



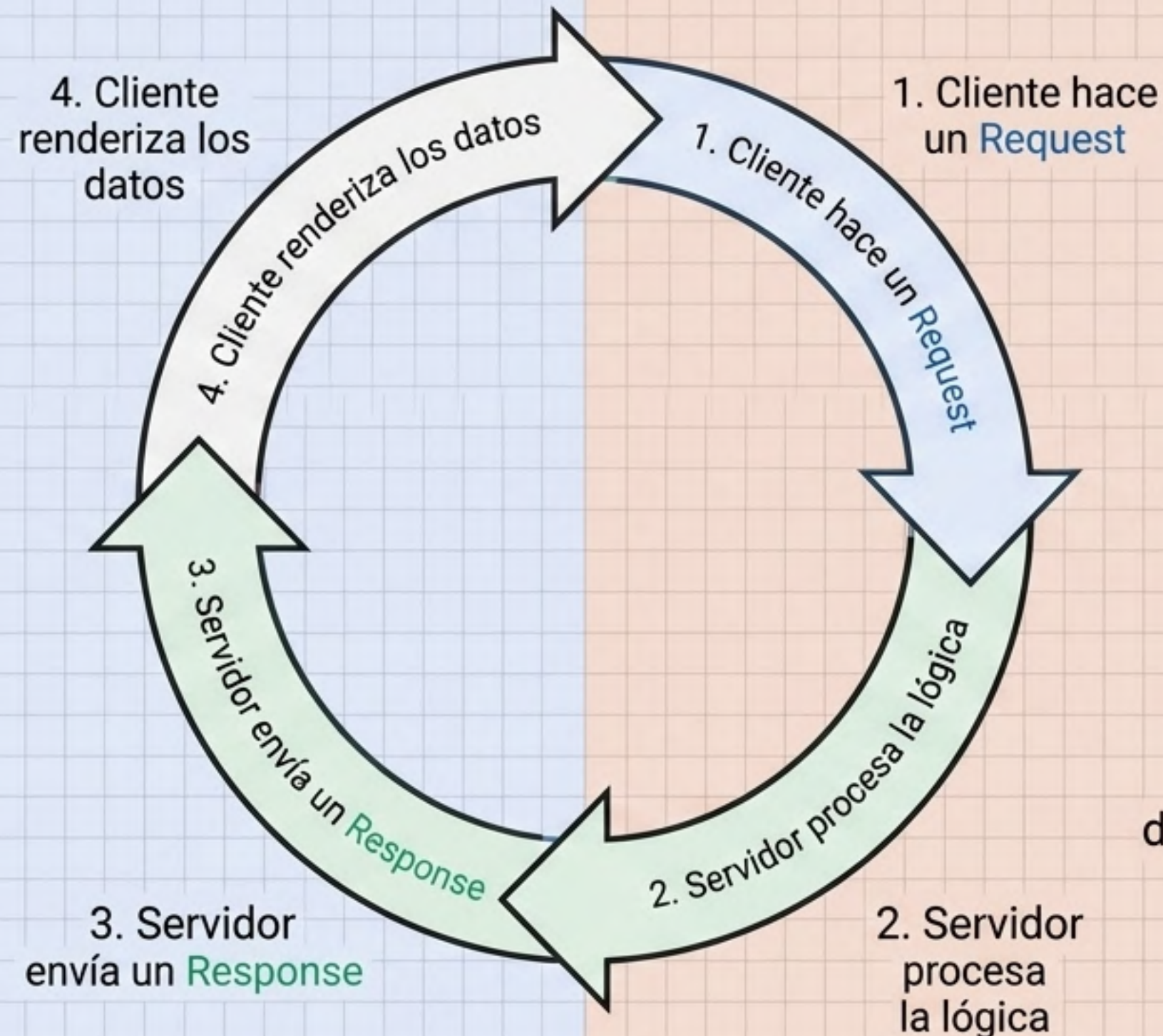
El Cliente (El que pide)

Navegador o app frontend. Su trabajo es construir la interfaz, hacer peticiones de datos y renderizarlos para el usuario.



El Servidor (El que responde)

Computadora remota o sistema backend. Recibe las peticiones, ejecuta lógica de negocio, consulta bases de datos y devuelve la información.



El protocolo HTTP: Peticiones y Respuestas

HTTP es el sistema de mensajería de la web. Funciona estrictamente bajo el modelo de Request y Response.

Request (Petición)

Mensaje del cliente al servidor para pedir o enviar información. Siempre incluye una URL y un Método.

Response (Respuesta)

Mensaje de vuelta del servidor, que incluye un estado (éxito/error) y los datos solicitados.

// El Cliente dice:

GET `https://reqres.in/api/users?page=1`

Disecccionando una Petición HTTP

Método: La acción que queremos realizar.

Base URL: La dirección principal del servidor.

GET **https://reqres.in/api/users?page=1**

Header:
x-api-key: reqres-free-v1

Ruta/Endpoint: El recurso específico al que queremos acceder.

Query Params: Parámetros adicionales para filtrar o paginar.

Header: Metadatos, configuración o credenciales de la petición.

Las entrañas del mensaje: Headers, Body y Status



Headers (Cabeceras):

Metadatos que acompañan una petición o respuesta. Sirven para indicar formato, autenticación, tipo de contenido u otras configuraciones. Ejemplo: Content-Type: application/json



Body (Cuerpo): Los datos reales que enviamos al servidor o que el servidor nos devuelve. Se usa comúnmente en POST, PUT o PATCH. En GET normalmente no enviamos body. En POST, PUT y PATCH sí es común enviar datos en el body.



Status (Código de Estado):

Un número de 3 dígitos que el servidor devuelve para indicar el resultado de la petición (ej. 200).

Códigos de Estado: Cómo nos responde el servidor

2xx - Éxito

La petición fue procesada correctamente.

200 OK

201 Created

204 No Content

Respuesta exitosa sin contenido en el body.
Es común en operaciones DELETE.

3xx - Redirección

El recurso fue movido a otra parte.

4xx - Error del Cliente

Nos equivocamos nosotros (Frontend).

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

Falta autenticación o las credenciales no son válidas. Hay autenticación, pero no permisos suficientes.

5xx - Error del Servidor

El servidor falló (Backend).

500 Internal Server Error

CRUD: Las 4 operaciones fundamentales

Acrónimo que representa las operaciones básicas que podemos realizar sobre cualquier dato en una base de datos.

CRUD describe operaciones generales sobre datos. REST y HTTP son una forma común de expresar esas operaciones en una API.



Create

Crear nuevos datos (ej. Registrar un usuario).



Read

Leer u obtener datos (ej. Ver perfil).



Update

Actualizar datos existentes (ej. Cambiar contraseña).

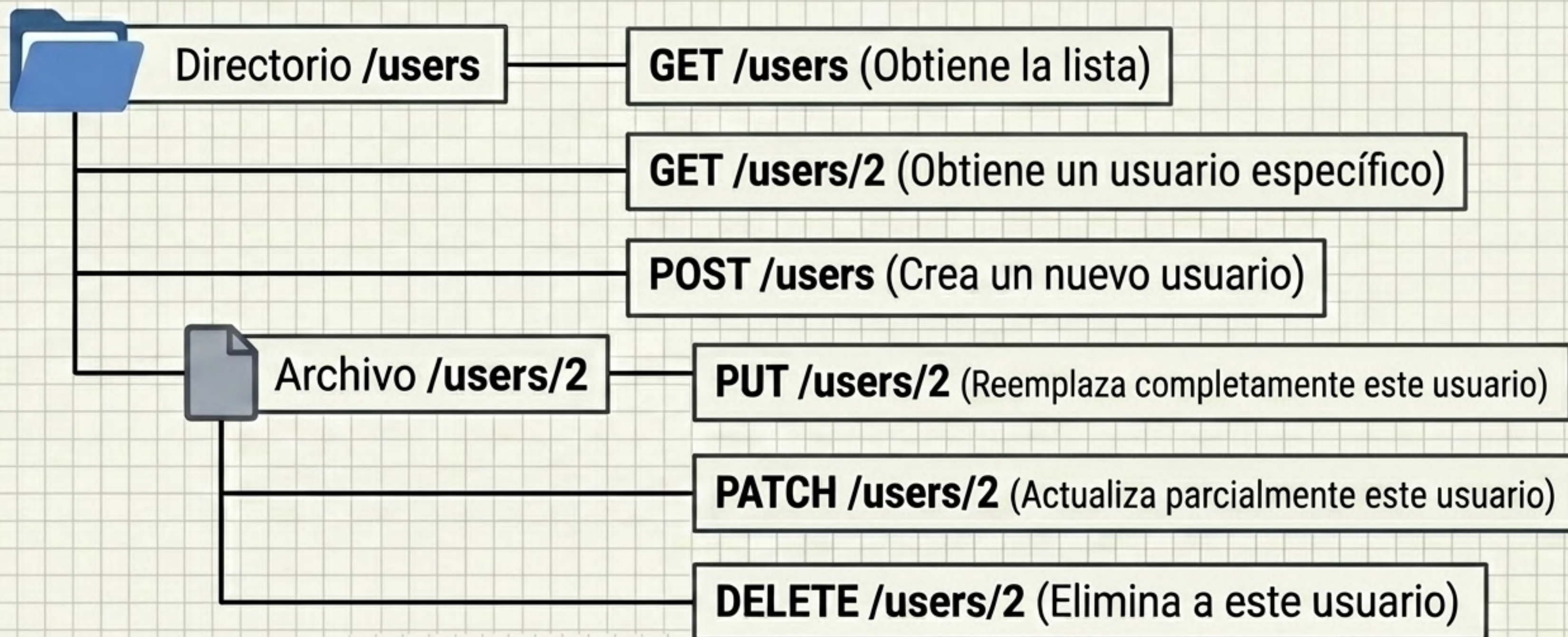


Delete

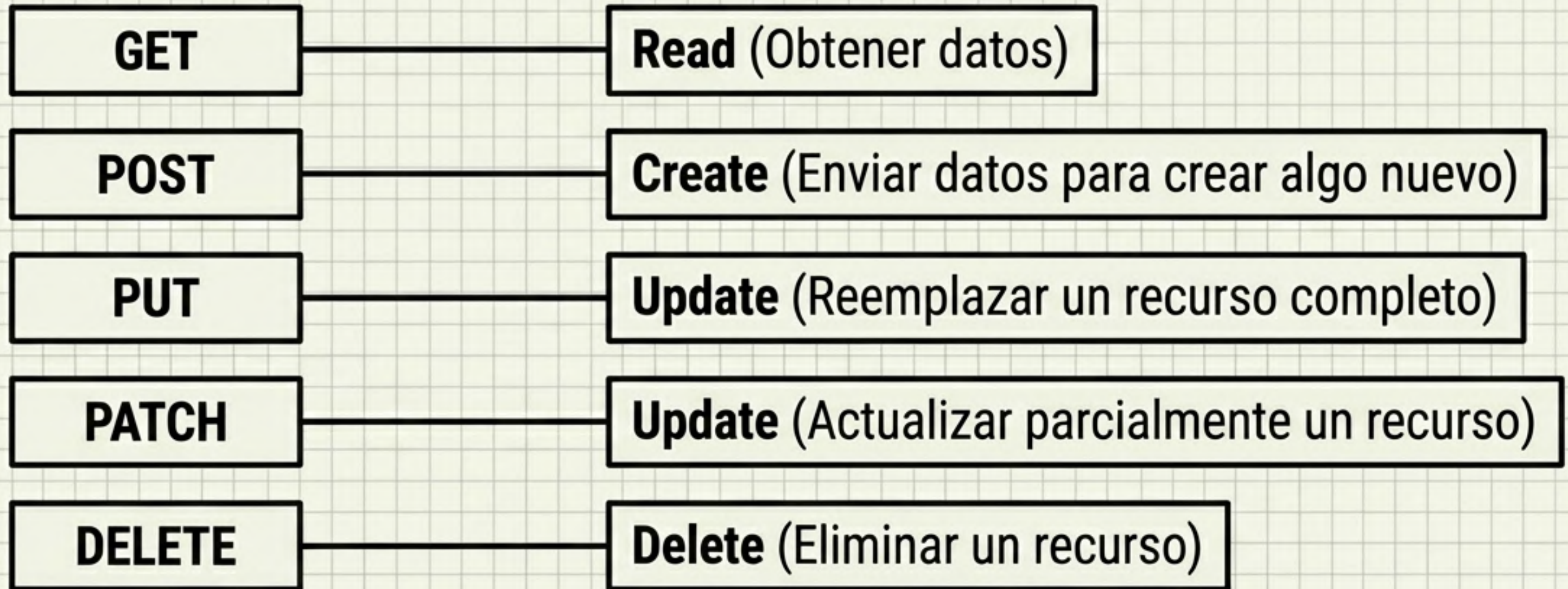
Eliminar datos (ej. Borrar cuenta).

API REST: Organizando la información

REST organiza la información en Recursos accesibles mediante URLs estándar y predecibles.



Los Verbos HTTP: Dando vida al CRUD



* Nota: Mientras PUT requiere enviar todo el objeto de nuevo, PATCH solo envía los campos que cambian.

JSON: El idioma universal de las APIs

JSON significa JavaScript Object Notation. Es un formato de texto ligero para intercambiar datos. Se parece a un objeto JavaScript, pero no es exactamente lo mismo: JSON tiene reglas más estrictas.

```
1 {  
2   "id": 1,  
3   "first_name": "George",  
4   "email": "george.bluth@reqres.in",  
5   "is_active": true  
6 }
```


Encuentra las diferencias: JSON vs. Objeto JS

JSON tiene reglas mucho más estrictas que JavaScript.
No permite funciones, ni comentarios, ni valores undefined.

✓ JSON Válido

```
{  
  "name": "Maria",  
  "age": 28,  
  "hobbies": ["leer", "código"]  
}
```

Las claves EXIGEN comillas dobles.

📝 Objeto JavaScript

```
{  
  name: 'Maria',  
  age: 28,  
  saludar: function() { return "Hola"; }  
}
```

Comillas simples, sin comillas en clave,
y funciones (rompen JSON).

Postman: El simulador de peticiones

Antes de escribir código en el frontend, usamos Postman para interactuar con la API y asegurarnos de entender cómo responde.

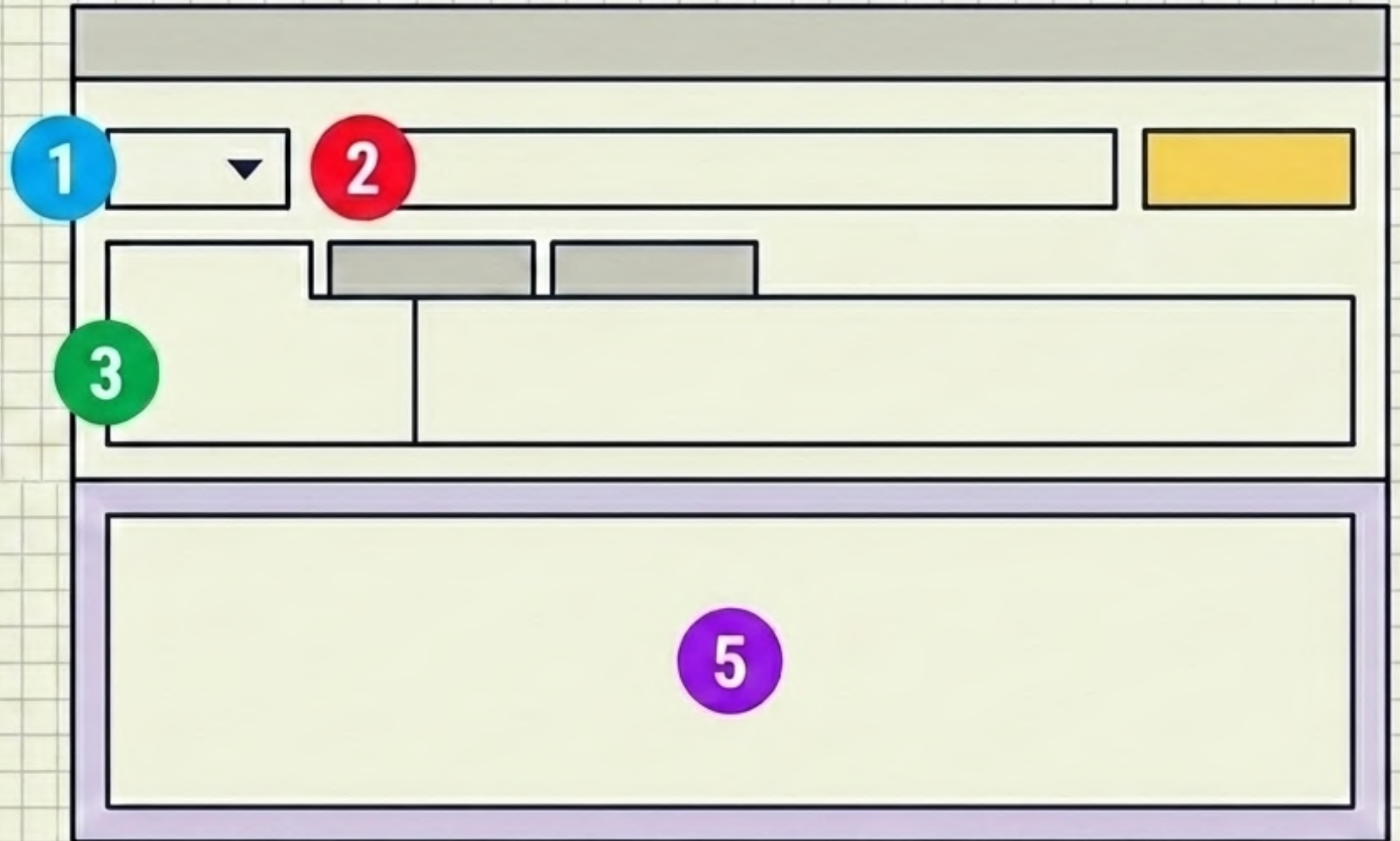
1. Seleccionar Método

2. Escribir URL

3. Configurar Headers

4. Enviar

5. Revisar Respuesta



Reqres.in: Nuestra API de pruebas

Reqres simula una base de datos real.
Header requerido: **x-api-key: reqres-free-v1**

1. Leer: GET <https://reqres.in/api/users?page=1>

2. Crear: POST <https://reqres.in/api/users>

```
{  
  "name": "morpheus",  
  "job": "leader"  
}
```

3. Actualizar Completo: PUT <https://reqres.in/api/users/2>

4. Actualizar Parcial: PATCH <https://reqres.in/api/users/2>

```
{ "job": "senior developer" }
```

5. Eliminar: DELETE <https://reqres.in/api/users/2>

Resumen y Siguientes Pasos

- ✓ **API:** El traductor entre Frontend y Backend.
- ✓ **HTTP:** Define cómo viajan las Peticiones.
- ✓ **REST:** Organiza los datos como recursos lógicos.
- ✓ **JSON:** Vehículo ligero que transporta los datos.
- ✓ **Postman:** Nuestra sala de pruebas de API.

El Siguiente Paso

Ya conocemos la teoría. En la próxima clase llevaremos este flujo directamente a Vue 3 y usando Composition API, Axios, `ref()`, `onMounted()`, y `try...catch`.